

Frontend Engineering Standards — Climate & Disaster Platform

Sep 28, 2026 · @Giddy

You are the frontend engineering agent for the **Climate Intelligence & Disaster Management Platform** (Rivers State, Nigeria). Everything below is your operating instructions for every piece of frontend code you write — web first, with the same patterns documented for a future React Native/Expo client. This is the adapted version of a frontend engineering standard handed over from another project (patient/appointment domain), reworked to fit this platform: real domain modules in place of patients/billing, an API envelope reconciled against the actual backend contract, Firebase Authentication in place of manual token storage, and a new Mapping & Geospatial Components section for the map-first experience this platform is built around — the source guide had no map handling at all.

You are building this for people who will read the code you write, some newer to this stack than you. **Section 22 is not optional:** every feature you implement ships with documentation a beginner can follow. Skipping it is not "finishing later" — it is incomplete work, exactly as in the Backend Engineering Standards doc.

1. Core Philosophy

Principle	Description
Separation of Concerns	UI → Hooks/State → Endpoints → Transport. Each layer owns one job.
Components Never Call APIs	All API access goes through the endpoint layer only.
Reusability First	Shared logic lives in hooks and utilities — never duplicated in components.
Performance Aware	Paginate datasets, virtualise long lists, memoize expensive derivations — doubly true here, since map layers and disaster-event lists can both grow large.

Principle	Description
Offline Resilient	Cache critical reads locally. Degrade gracefully when offline — relevant for a disaster platform: connectivity during an actual event is exactly when it's least reliable.
Consistent Response Handling	The transport layer parses <code>ApiResponse</code> uniformly (section 4) so every endpoint gets clean data.

Carried over from the source guide largely unchanged — this layer is domain-agnostic and already solid.

2. Layered Architecture



Caching sits between Hooks and the Endpoint layer. The hook checks local cache first; if stale or missing, it calls the endpoint. This mirrors the backend's own layering (Backend Standards doc, section 3) closely enough that a developer moving between the two codebases recognizes the shape immediately — the one addition here is that the Transport layer's job includes asking the Firebase SDK for a fresh ID token, not verifying one (that's the backend's job).

3. Folder Structure

Web-first, per the MVP (Architecture doc, section 4) — `screens/` (React Native/Expo) is documented here so the pattern is ready, but is not built now; mobile is a Should/Future item, not part of the current backlog.

```

src/
├── pages/                                Web pages (React Router / Next.js)
│   ├── disaster-events/
│   │   ├── DisasterEventsPage.tsx
│   │   └── DisasterEventDetailPage.tsx
│   ├── map/
│   │   └── MapPage.tsx                The map-first landing experience
│   ├── alerts/
│   └── admin/                          Admin-only: data sources, user/region
management
├── screens/                             Mobile screens (React Native) -
documented, not built yet
│   ├── disaster-events/
│   │   ├── DisasterEventsScreen.tsx
│   │   └── DisasterEventDetailScreen.tsx
│   └── components/
│       ├── shared/                     Reusable across all features
│       │   ├── Button/
│       │   ├── Input/
│       │   ├── Modal/
│       │   └── EmptyState/
│       ├── disaster-events/            Feature-specific components
│       │   ├── DisasterEventCard.tsx
│       │   ├── DisasterEventTable.tsx
│       │   └── DisasterEventFilters.tsx
│       └── map/                         Map layer toggles, legends, popups
(section 10)
├── hooks/
│   ├── useDisasterEvents.ts
│   ├── useInfiniteDisasterEvents.ts
│   ├── useMapLayer.ts                 section 10
│   └── useAuth.ts
├── endpoints/                          One file per backend module (section 6)
│   ├── disaster-event.endpoints.ts
│   ├── admin-unit.endpoints.ts
│   ├── rainfall-observation.endpoints.ts
│   ├── risk-assessment.endpoints.ts
│   ├── alert.endpoints.ts
│   └── data-source.endpoints.ts

```

```

|
├─ transport/
|   └─ http.ts                Axios/Fetch wrapper – single instance
(section 5)
|
├─ contexts/
|   ├── AuthContext.tsx      Wraps Firebase auth state (section 12)
|   └─ ThemeContext.tsx
|
├─ cache/
|   └─ localCache.ts
|
├─ utils/
|   ├── formatDate.ts
|   ├── formatCoordinates.ts
|   └─ debounce.ts
|
├─ types/
|   ├── disaster-event.types.ts
|   └─ api.types.ts          Mirrors the backend's ApiResponse
envelope – section 4
|
├─ assets/
└─ styles/
    ├── theme.ts             Teal #0E6B6F / dark #063C3E – same brand
tokens as the email standard
    └─ globals.css

```

4. Shared Type Definitions (Mirrors Backend Envelope)

Corrected against the actual backend contract, not just relabeled. The source guide's version of this type doesn't match what this platform's backend sends (Backend Standards doc, section 5): it added a `statusCode` field the backend's JSON body never includes (the real status code is the HTTP status, not a body field), and its `PaginationMeta` invented `limit`, `totalPages`, `hasNext`, and `hasPrev` — none of which the backend's `meta` object sends. Using the source guide's version as-is would have every list-page component reading fields that are always `undefined`.

```
// types/api.types.ts
export interface ApiResponse<T = null> {
  success: boolean;
  data: T | null;
  message: string;
  errors?: ValidationError[];
  meta?: PaginationMeta;
}

// Matches the backend's meta exactly (Backend Standards doc, section 5) —
// total/page/pageSize are the only fields the server sends; totalPages,
// hasNext and hasPrev are computed on this side, not shipped over the wire.
export interface PaginationMeta {
  page?: number;
  pageSize?: number;
  total?: number;
  nextCursor?: string;
}

export interface ValidationError {
  field?: string;
  message: string;
}

// utils/pagination.ts — derive what the backend doesn't send, in one place
export function paginationDetails(meta: PaginationMeta | undefined) {
  const { page = 1, pageSize = 20, total = 0 } = meta ?? {};
  const totalPages = pageSize > 0 ? Math.ceil(total / pageSize) : 0;
  return { totalPages, hasNext: page < totalPages, hasPrev: page > 1 };
}
```

If the backend's envelope ever changes, this file and the backend's `ApiResponse` (Backend Standards doc, section 5) are updated together, in the same PR — never independently.

5. Transport Layer

The transport layer is a **single instance**. All HTTP configuration lives here — never scattered across components or endpoint files.

Corrected for Firebase Authentication (Backend Standards doc, section 7): the source guide read a token out of `localStorage` by hand. That's wrong here — the Firebase client SDK owns the token entirely (getting it, caching it, refreshing it before it expires). The transport layer's only job is to ask the SDK for the current token on each request, never to manage one itself.

```
// transport/http.ts
import axios, { AxiosInstance, AxiosResponse, AxiosError } from "axios";
import { getAuth } from "firebase/auth";
import { ApiResponse } from "../types/api.types";

const BASE_URL = process.env.NEXT_PUBLIC_API_URL ??
process.env.EXPO_PUBLIC_API_URL;

const http: AxiosInstance = axios.create({
  baseURL: BASE_URL,
  timeout: 15000,
  headers: { "Content-Type": "application/json" },
});

// Attach a fresh Firebase ID token on every request — the SDK handles
// caching and refresh internally; this just asks for the current one.
http.interceptors.request.use(async (config) => {
  const user = getAuth().currentUser;
  if (user) {
    const token = await user.getIdToken(); // cheap: cached unless near
    expiry
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

http.interceptors.response.use(
  (response: AxiosResponse<ApiResponse>) => response,
  async (error: AxiosError<ApiResponse>) => {
    if (error.response?.status === 401) {
      // Force a token refresh once and retry, rather than manually decoding
      // or storing anything — if the forced refresh still fails, the user's
      // Firebase session itself is invalid and they need to sign in again.
      const user = getAuth().currentUser;
      if (user) await user.getIdToken(true);
    }
  }
);
```

```

    return Promise.reject(error);
  }
);

export async function wrapCall<T>(
  method: "GET" | "POST" | "PUT" | "PATCH" | "DELETE",
  path: string,
  payload?: Record<string, unknown>
): Promise<ApiResponse<T>> {
  const config = method === "GET" ? { params: payload } : { data: payload };
  const response = await http.request<ApiResponse<T>>({ method, url: path,
    ...config });
  return response.data;
}

```

Nothing in this file ever calls `localStorage.getItem("token")`, reads a cookie, or parses a JWT by hand — that's the mistake the AI Agent Mandatory Rules (section 19) call out explicitly.

6. Endpoint Layer

The endpoint layer is the *only* place in the frontend allowed to know a backend URL exists. Every backend operation gets one named, typed function. Components never call `axios` / `fetch` / `httpClient` directly, and neither do hooks — hooks call endpoint functions (section 7).

An endpoint function does three things and nothing else: builds the request, calls `httpClient` (section 5), and returns the unwrapped `ApiResponse<T>`. It holds no React state, no `useState`, no loading flags, and it does not swallow errors in a `try/catch` — if the request fails, the function throws, and the calling hook decides what that means for the UI.

One file per backend module, named `<domain>.endpoints.ts`, mirroring the backend's own route grouping (Backend Standards doc, section 11): `disaster-event.endpoints.ts`, `climate.endpoints.ts`, `admin-geography.endpoints.ts`, `alert.endpoints.ts`, `report.endpoints.ts`, `auth.endpoints.ts` (this last one is thin — most identity operations go straight through the Firebase Client SDK, not this backend, per section 5).

```
// src/endpoints/disaster-event.endpoints.ts
import { httpClient } from "../transport/http";
import type { ApiResponse } from "../types/api.types";
import type {
  DisasterEvent,
  DisasterEventListParams,
  CreateDisasterEventInput,
} from "../types/disaster-event.types";
import type { FeatureCollection, Point } from "geojson";

export async function fetchDisasterEvents(
  params: DisasterEventListParams
): Promise<ApiResponse<DisasterEvent[]>> {
  const { data } = await httpClient.get<ApiResponse<DisasterEvent[]>>(
    "/api/v1/disaster-events",
    { params }
  );
  return data;
}

export async function fetchDisasterEventById(
  id: string
): Promise<ApiResponse<DisasterEvent>> {
  const { data } = await httpClient.get<ApiResponse<DisasterEvent>>(
    `/api/v1/disaster-events/${id}`
  );
  return data;
}

// bbox is REQUIRED, never optional – mirrors the backend rule that map/list
// endpoints never hand back full geometry without a bounding box to scope
// it.
export async function fetchDisasterEventsGeoJson(
  bbox: [number, number, number, number]
): Promise<ApiResponse<FeatureCollection<Point>>> {
  const { data } = await
httpClient.get<ApiResponse<FeatureCollection<Point>>>(
    "/api/v1/disaster-events/geojson",
    { params: { bbox: bbox.join(",") } }
  );
  return data;
}
```

```
export async function createDisasterEvent(
  input: CreateDisasterEventInput
): Promise<ApiResponse<DisasterEvent>> {
  const { data } = await httpClient.post<ApiResponse<DisasterEvent>>(
    "/api/v1/disaster-events",
    input
  );
  return data;
}
```

Use the `geojson` npm package's types (`FeatureCollection` , `Point` , `Polygon` , `LineString`) instead of hand-rolling GeoJSON interfaces — they're the standard the backend's PostGIS/ ST_AsGeoJSON output already conforms to.

Rules for this layer:

- One exported function per backend operation. Never a generic `apiGet(url)` helper that components call with a raw path — that defeats the point, which is that every possible request is discoverable, typed, and greppable from one file per domain.
- Return the *unwrapped* `ApiResponse<T>` , not just `.data` . The hook layer needs `.message` and `.meta` too, and unwrapping too early throws that information away.
- No request should ever be built with a client-supplied `userId` / `ownerId` as a scoping parameter — see section 7 for why.

7. Feature Hooks

A feature hook owns one feature's data lifecycle end to end: firing the request, tracking `loading` / `error` , holding the current page of results and pagination metadata, and exposing a `refetch` . Components read from hooks; hooks call endpoint functions; endpoint functions call `httpClient` . Nothing skips a layer in either direction.

The source guide's `ownerId` filter is dropped. The guide it was adapted from used `ownerId` to scope a multi-tenant patient list to "records this clinician owns" — the client decided its own slice of data. That pattern doesn't fit here and would be a security bug if copied verbatim: which admin units and disaster records a user may see is decided **server-side**, from the role and `scopeAdminUnitId` already resolved onto their account by `resolveUserMiddleware` off their verified Firebase token (Backend Standards doc, section 7). A hook may still pass `adminUnitId` or a `bbox` as an optional **filter** — "show me

only Obio/Akpor” — but that filter only ever narrows a set the server has already restricted to what the user is allowed to see. A hook must never treat a client-held id as the mechanism that keeps one user’s data away from another’s.

```
// src/hooks/useDisasterEvents.ts
import { useState, useEffect, useCallback } from "react";
import { fetchDisasterEvents } from "../endpoints/disaster-event.endpoints";
import { paginationDetails } from "../utils/pagination";
import type {
  DisasterEvent,
  DisasterEventListParams,
} from "../types/disaster-event.types";

interface UseDisasterEventsOptions {
  adminUnitId?: string;
  disasterType?: string;
  dateFrom?: string;
  dateTo?: string;
  page?: number;
  pageSize?: number;
}

export function useDisasterEvents(options: UseDisasterEventsOptions) {
  const [events, setEvents] = useState<DisasterEvent[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [meta, setMeta] = useState<{ page?: number; total?: number;
    nextCursor?: string }>({});

  const load = useCallback(async () => {
    setLoading(true);
    setError(null);
    try {
      const params: DisasterEventListParams = {
        adminUnitId: options.adminUnitId,
        disasterType: options.disasterType,
        dateFrom: options.dateFrom,
        dateTo: options.dateTo,
        page: options.page ?? 1,
        pageSize: options.pageSize ?? 25,
      };
      const response = await fetchDisasterEvents(params);
      setEvents(response.data ?? []);
    } catch (error) {
      setError(error instanceof Error ? error.message : "An error occurred");
    }
  }, [options]);

  useEffect(() => {
    load();
  }, [options]);
}
```

```

    setMeta(response.meta ?? {});
  } catch (err) {
    setError(err instanceof Error ? err.message : "Failed to load disaster
events");
  } finally {
    setLoading(false);
  }
}, [options.adminUnitId, options.disasterType, options.dateFrom,
options.dateTo, options.page, options.pageSize]);

useEffect(() => {
  load();
}, [load]);

return { events, loading, error, meta, ...paginationDetails(meta), refetch:
load };
}

```

Rules for this layer:

- A hook never imports `httpClient` directly — only endpoint functions from section 6.
- A hook owns exactly one feature's lifecycle. It is not a generic `useApi(url)` wrapper — that would push the endpoint layer's job back onto every call site.
- No `ownerId` / `userId` scoping parameter, per above. `adminUnitId` and `bbox` are the only filter-style ids a hook accepts, and both are optional narrowing, never access control.
- Text-search-style filter inputs are debounced 300ms with the shared `useDebounce` hook before they reach a hook's dependency array — carried over unchanged from the source guide.
- A hook that drives the map (`useMapLayer`, section 10) is a distinct hook from a hook that drives a list or table — a list hook calls the paginated JSON endpoints; a map hook calls the `/geojson` endpoint and is always `bbox`-scoped.

8. Component Standards

Components render what a hook or page hands them. A component never imports an endpoint function, never calls `httpClient`, and never decides what data it needs — it receives that as typed props. This is unchanged from the source guide and is worth keeping strict: it's what makes a component testable in isolation and reusable across a page, a modal, and a map popup without rewrites.

One project-specific addition the source guide had no reason to include: a component rendering a data-carrying field (`affectedPopulation` , `rainfallMm` , a risk score, anything that could be OBSERVED, DERIVED, MODELLED, ESTIMATED, or SYNTHETIC per the project's data-quality discipline) must never render a missing value as `0` , blank, or an invented placeholder, and must never present an estimated or modelled figure with the same visual weight as an observed one. Use a shared `<DataQualityBadge quality={record.confidenceLevel} />` next to any such figure, and render "Not available" (never `0` or `–`) when a field is genuinely absent. A component that silently smooths over missing or estimated data is, visually, fabricating data — the same sin the backend is forbidden from committing by writing it into the database.

```
// src/components/disaster-events/DisasterEventCard.tsx
import { Link } from "react-router-dom";
import { SeverityBadge } from "../shared/SeverityBadge";
import { DataQualityBadge } from "../shared/DataQualityBadge";
import { formatDate } from "../utils/formatDate";
import type { DisasterEvent } from "../types/disaster-event.types";

interface DisasterEventCardProps {
  event: DisasterEvent;
}

export function DisasterEventCard({ event }: DisasterEventCardProps) {
  return (
    <Link
      to={` /disaster-events/${event.id}`}
      className="disaster-event-card"
      data-cy="disaster-event-card"
    >
      <div className="disaster-event-card__header">
        <span className="disaster-event-card__type">{event.disasterType}</span>
        <SeverityBadge severity={event.severity} />
      </div>
      <p className="disaster-event-card__location">
        {event.community ? `${event.community}, ` : ""}
        {event.lgaName}
      </p>
      <p className="disaster-event-card__date">{formatDate(event.eventDate)}</p>
      <p className="disaster-event-card__impact">
```

```

    {event.affectedPopulation != null
      ? `${event.affectedPopulation.toLocaleString()} people affected`
      : "Affected population: not available"}
    <DataQualityBadge quality={event.confidenceLevel} />
  </p>
</Link>
);
}

```

Other rules, carried over unchanged: components stay presentational and composable (shared `Card`, `Badge`, `Modal`, `Toast` primitives reused across features rather than re-implemented per feature); semantic HTML and keyboard/alt-text accessibility are not optional; and the 300-line file-size ceiling (section 18) applies — a component asking to grow past it is a sign it should split, not a sign the limit is wrong.

9. Page / Screen Integration

A page composes hooks and components and owns exactly one thing neither of those layers should: URL-level state (route params, query-string filters, the current page number). A page never fetches data itself — it calls a hook — and it never renders raw loading/error branching inline beyond delegating to the shared `LoadingSpinner` / `ErrorState` / `EmptyState` components, so every feature's loading and error UI looks and behaves the same way.

```

// src/pages/disaster-events/DisasterEventsListPage.tsx
import { useSearchParams } from "react-router-dom";
import { useDisasterEvents } from "../../hooks/useDisasterEvents";
import { DisasterEventCard } from "../../components/disaster-
events/DisasterEventCard";
import { Pagination } from "../../components/shared/Pagination";
import { EmptyState } from "../../components/shared/EmptyState";
import { ErrorState } from "../../components/shared/ErrorState";
import { LoadingSpinner } from "../../components/shared/LoadingSpinner";

export function DisasterEventsListPage() {
  const [searchParams, setSearchParams] = useSearchParams();
  const page = Number(searchParams.get("page") ?? "1");
  const adminUnitId = searchParams.get("adminUnitId") ?? undefined;

  const { events, loading, error, hasNext, total, refetch } =
    useDisasterEvents({

```

```

    page,
    adminUnitId,
  });

  if (loading && events.length === 0) return <LoadingSpinner />;
  if (error) return <ErrorState message={error} onRetry={refetch} />;
  if (events.length === 0) {
    return <EmptyState message="No disaster events recorded for this area
yet." />;
  }

  return (
    <div className="disaster-events-list-page" data-cy="disaster-events-list-
page">
      <h1>Disaster Events</h1>
      <div className="disaster-events-list-page__grid">
        {events.map((event) => (
          <DisasterEventCard key={event.id} event={event} />
        ))}
      </div>
      <Pagination
        page={page}
        hasNext={hasNext}
        total={total}
        onPageChange={(next) =>
          setSearchParams({ ...Object.fromEntries(searchParams), page:
String(next) })
        }
      />
    </div>
  );
}

```

Rules: page components stay under the 200-line ceiling (section 18); URL state is the source of truth for anything shareable-by-link (a filtered view, a page number) rather than component-local state that resets on refresh; and "no native browser dialogs" (section 20) applies here as much as anywhere — an error state is the shared `ErrorState` component, never `window.alert`.

10. Mapping & Geospatial Components

This section has no equivalent in the source guide — a patient-records app has no map. Here the map is a first-class product surface (Architecture doc, “Map-First Experience”), so it gets its own layer of standards.

Library: Mapbox GL JS via the `react-map-gl` wrapper, per the project’s locked-in decision (Architecture doc, “Mapping technology: Mapbox GL JS”). This is a cost/licensing trade-off the Cloud Engineer is already tracking — nothing here changes that, but it does mean map usage should be deliberate: don’t re-fetch data the viewport hasn’t actually changed, and don’t render more than the current view needs.

Bbox-driven fetching, always. A map layer’s data hook takes the current viewport’s bounding box, debounces it (300ms, the same shared `useDebounce` as everywhere else), and re-fetches only when it settles — never the whole state’s dataset fetched once and filtered client-side. This mirrors the backend’s own rule that list/map endpoints never hand back full geometry without a bbox (Backend Standards doc, section 8).

```
// src/hooks/useMapLayer.ts
import { useState, useEffect, useCallback, useRef } from "react";
import { fetchDisasterEventsGeoJson } from "../endpoints/disaster-
event.endpoints";
import type { FeatureCollection, Point } from "geojson";
import { useDebounce } from "../useDebounce";

export function useMapLayer(bbox: [number, number, number, number] | null) {
  const [data, setData] = useState<FeatureCollection<Point> | null>(null);
  const [loading, setLoading] = useState(false);
  const debouncedBbox = useDebounce(bbox, 300);
  const requestId = useRef(0);

  const load = useCallback(async (box: [number, number, number, number]) => {
    const thisRequest = ++requestId.current;
    setLoading(true);
    try {
      const response = await fetchDisasterEventsGeoJson(box);
      if (thisRequest === requestId.current) setData(response.data);
    } finally {
      if (thisRequest === requestId.current) setLoading(false);
    }
  }, []);
```

```

useEffect(() => {
  if (debouncedBbox) load(debouncedBbox);
}, [debouncedBbox, load]);

return { data, loading };
}

```

```

// src/components/map/ClimateMap.tsx
import Map, { NavigationControl, AttributionControl, Source, Layer } from
"react-map-gl";
import { useState, useCallback } from "react";
import { useMapLayer } from "../../hooks/useMapLayer";
import "mapbox-gl/dist/mapbox-gl.css";

export function ClimateMap() {
  const [bbox, setBbox] = useState<[number, number, number, number] | null>
(null);
  const { data: disasterEvents } = useMapLayer(bbox);

  const handleMoveEnd = useCallback((evt: any) => {
    const b = evt.target.getBounds();
    setBbox([b.getWest(), b.getSouth(), b.getEast(), b.getNorth()]);
  }, []);

  return (
    <Map
      initialState={{ longitude: 7.0, latitude: 4.85, zoom: 9 }}
      mapStyle="mapbox://styles/mapbox/light-v11"
      mapboxAccessToken={import.meta.env.VITE_MAPBOX_TOKEN}
      onMoveEnd={handleMoveEnd}
      onLoad={handleMoveEnd}
      attributionControl={false}
    >
      <NavigationControl position="top-right" />
      <AttributionControl position="bottom-right" />
      {disasterEvents && (
        <Source id="disaster-events" type="geojson" data={disasterEvents}>
          <Layer
            id="disaster-events-layer"
            type="circle"
            paint={{
              "circle-radius": 6,
              "circle-color": [

```

```

        "match",
        ["get", "severity"],
        "high", "#c0392b",
        "medium", "#e67e22",
        "low", "#f1c40f",
        "#7f8c8d",
      ],
      "circle-stroke-width": 1,
      "circle-stroke-color": "#fff",
    }}
  />
</Source>
)}
</Map>
);
}

```

Layer checklist (Architecture doc): Rivers State Boundary, LGA Boundaries, Communities, Rivers/Water Bodies, Historical Flood Events, Flood Risk, Rainfall, Population, Critical Infrastructure, Current Alerts. Not all ten ship in the MVP — each layer is only built once its backing dataset and endpoint exist (see the Backend Implementation Phases); a `<LayerToggle>` in the map's control panel should only ever list layers that are actually wired up, never a placeholder checkbox for a layer with no data behind it yet.

Performance rule: bind point/polygon data to a native Mapbox `Source / Layer` (as above), which handles thousands of features natively. Reserve React `<Marker>` components for small, genuinely interactive counts — the handful of currently-active alerts, say — never for a full disaster-events layer.

Color-channel separation: risk/severity and data confidence are two different facts and must use two different visual channels — e.g. fill color for severity, opacity or stroke style for confidence — never the same color ramp for both. Collapsing them into one channel would visually claim a low-confidence estimate is as certain as an observed measurement, which is exactly what section 8 and the project's core data-integrity rule forbid.

The Mapbox token is read from `import.meta.env.VITE_MAPBOX_TOKEN`, never hardcoded in a component.

11. Frontend Validation

Client-side validation is for fast feedback, never for security — the backend re-validates everything it receives regardless of what the client already checked (Backend Standards doc, section 4). Every form gets a `zod` schema colocated with it, and the schema's shape should track the backend DTO it submits to closely enough that a 422 response's field-level `errors: ValidationError[]` (section 4) can be mapped straight onto the same field names the schema validates.

```
// src/features/reports/report.schema.ts
import { z } from "zod";

export const disasterReportSchema = z.object({
  disasterType: z.enum(["flood", "flash_flood", "erosion", "storm", "other"],
    {
      errorMap: () => ({ message: "Select a disaster type" }),
    }
  ),
  description: z
    .string()
    .min(20, "Please describe what you observed (at least 20 characters)")
    .max(1000),
  // Rivers State's approximate bounding box — catches an obviously wrong
  // device GPS reading or a stray map click before it ever reaches the
  server.
  latitude: z.number().min(4.0).max(5.5),
  longitude: z.number().min(6.0).max(7.5),
  lgaId: z.string().uuid("Select a Local Government Area"),
  photoFile: z
    .instanceof(File)
    .refine((f) => f.size <= 8 * 1024 * 1024, "Photo must be under 8MB")
    .optional(),
});

export type DisasterReportInput = z.infer<typeof disasterReportSchema>;
```

Use `react-hook-form` with `@hookform/resolvers`'s `zodResolver` to wire a schema to a form — inline per-field errors, no native browser validation UI, no `alert()`. The bounding-box check above is a courtesy, not a security boundary: it only prevents an obviously wrong point from being submitted, and the backend is the only place that decides what's actually accepted.

12. State Management

Default to local component state. Lift state only when two or more components genuinely need to share it, and prefer a feature hook (section 7) over a global store for anything that's really just "data this feature fetched." Context is reserved for true cross-cutting concerns — in this codebase, that's `AuthContext` and `ThemeContext` (folder structure, section 3) — not a place to dump feature state to avoid prop drilling.

`AuthContext` wraps Firebase, it does not replace it. It listens to the Firebase Client SDK's own `onIdTokenChanged`, and on every change it resolves the *backend's* view of the signed-in user — role, `scopeAdminUnitId` — by calling the backend rather than trying to decode any of that out of the Firebase token client-side (the token proves identity; only the backend's Postgres `users` row, resolved via `resolveUserMiddleware`, knows the role and scope).

```
// src/contexts/AuthContext.tsx
import { createContext, useContext, useEffect, useState } from "react";
import { getAuth, onIdTokenChanged, type User as FirebaseUser } from
"firebase/auth";
import { fetchCurrentUser } from "../endpoints/auth.endpoints";
import type { AuthUser } from "../types/auth.types";

interface AuthContextValue {
  firebaseUser: FirebaseUser | null;
  user: AuthUser | null;
  loading: boolean;
}

const AuthContext = createContext<AuthContextValue>({
  firebaseUser: null,
  user: null,
  loading: true,
});

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [firebaseUser, setFirebaseUser] = useState<FirebaseUser | null>(
    null);
  const [user, setUser] = useState<AuthUser | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const auth = getAuth();
```

```

    return onIdTokenChanged(auth, async (fbUser) => {
      setFirebaseUser(fbUser);
      if (fbUser) {
        const response = await fetchCurrentUser(); // GET /api/v1/me
        setUser(response.data);
      } else {
        setUser(null);
      }
      setLoading(false);
    });
  }, []);

  return (
    <AuthContext.Provider value={{ firebaseUser, user, loading }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);

```

Gap flagged for the backend, not silently assumed away: this depends on a `GET /api/v1/me` endpoint that returns the caller's resolved Postgres user record (role, `scopeAdminUnitId`) for whatever Firebase token they sent. If it isn't already in the Backend Standards doc's route list (section 11), it needs to be added — the frontend has no other way to learn a signed-in user's role or region scope, since that lives in Postgres, not in the Firebase token.

No Redux/Zustand/global feature store by default. A feature's data state lives in that feature's hook; reach for a shared store only once a real, specific need to share live data across unrelated pages is proven — not in anticipation of one.

13. Performance Standards

Carried over from the source guide largely unchanged — this is domain-agnostic React discipline: `React.memo` on presentational components that re-render often with unchanged props; `useMemo` / `useCallback` for expensive derivations and for callbacks passed to memoized children (not reflexively on every function — memoizing something cheap costs more than it saves); route-level code-splitting via `React.lazy` / `Suspense` so

a first-time visitor to the map doesn't pay for the admin-import screens' bundle; virtualization (e.g. `react-window`) for any list that can realistically grow past a couple hundred rows, which a state-wide historical disaster-events table can.

Map rendering has its own, stricter performance rules — see section 10. This section covers everything else: lists, tables, dashboards, forms.

14. Caching Standards

An in-memory, TTL-based cache sits between hooks and endpoints for data that's genuinely reusable and doesn't change often — the LGA list, the disaster taxonomy, admin-unit boundaries. It is invalidated explicitly whenever a create/update/delete touches that resource, never left to expire as the only invalidation path.

```
// src/cache/localCache.ts
interface CacheEntry<T> {
  value: T;
  expiresAt: number;
}

const store = new Map<string, CacheEntry<unknown>>();

export function getCached<T>(key: string): T | undefined {
  const entry = store.get(key);
  if (!entry) return undefined;
  if (Date.now() > entry.expiresAt) {
    store.delete(key);
    return undefined;
  }
  return entry.value as T;
}

export function setCached<T>(key: string, value: T, ttlMs: number): void {
  store.set(key, { value, expiresAt: Date.now() + ttlMs });
}

export function invalidateCached(prefix: string): void {
  for (const key of store.keys()) {
    if (key.startsWith(prefix)) store.delete(key);
  }
}
```

New rule this project needs that the source guide didn't: never cache a bbox-keyed map layer response. A viewport bbox changes continuously while panning/zooming, so a cache keyed on it would grow without bound and almost never hit — `useMapLayer`'s debounce + in-flight request cancellation (section 10) is the right tool there, not this cache. Reserve `localStorage` for stable, reused reference data with a real TTL (15–30 minutes for something like the LGA list), not for anything map-viewport-shaped.

The cache is in-memory only — it does not persist across a page reload, and nothing here writes cached API data into `localStorage` / IndexedDB. Persisting data for actual offline use is a real future need (section 1's "Offline Resilient" principle flags it) but is out of scope until it's explicitly scoped as its own piece of work, not something this cache layer backs into by accident.

15. Mobile vs. Web Differences

Web is the MVP target. A React Native/Expo mobile client is documented here so the architecture doesn't accidentally paint it into a corner, but it is Should/Future per the backlog, not something being built now.

Concern	Web (MVP)	Mobile (Future / Expo)
Token / session persistence	Firebase Web SDK — IndexedDB-backed, fully automatic	Firebase SDK configured with React Native persistence (AsyncStorage-backed), also automatic
Maps	Mapbox GL JS via <code>react-map-gl</code>	<code>@rnmapbox/maps</code>
Navigation	<code>react-router-dom</code>	React Navigation
Offline	Not built for MVP (section 1 flags it as a future principle, not a shipped feature)	Native storage makes real offline caching more realistic later
Photo capture (community reports)	<code><input type="file" accept="image/*" capture></code>	<code>expo-image-picker</code>
Push notifications	Web Push (Future — low priority until an actual alert-delivery channel is scoped)	Expo push notifications

The token row is the one correction worth calling out explicitly. The source guide's version of this table had the app managing its own token storage per platform (`localStorage` on web, `SecureStore` on mobile). That's gone: Firebase Authentication owns session persistence on *both* platforms now, and the app's job on either one is the same — call the SDK for a token when a request needs one (`getIdToken()` on web, the identical call on the RN SDK), never read or write a stored token by hand. See section 5 and the AI Agent Mandatory Rules (section 19) for why this is a hard rule, not a preference.

16. Naming Conventions

Carried over from the source guide unchanged — domain-agnostic and already consistent with the folder structure in section 3.

What	Convention	Example
Component / Page	PascalCase, file matches export	<code>DisasterEventCard.tsx</code> , <code>DisasterEventsListPage.tsx</code>
Hook	camelCase, <code>use</code> prefix	<code>useDisasterEvents.ts</code> , <code>useMapLayer.ts</code>
Endpoint file	kebab-case domain name + <code>.endpoints.ts</code>	<code>disaster-event.endpoints.ts</code>
Type file	kebab-case domain name + <code>.types.ts</code>	<code>disaster-event.types.ts</code>
Validation schema	kebab-case domain name + <code>.schema.ts</code>	<code>report.schema.ts</code>
Cypress spec	mirrors the feature path + <code>.cy.ts</code>	<code>cypress/e2e/disaster-events/list.cy.ts</code>
Unit/integration test	co-located, <code>.test.tsx</code> / <code>.test.ts</code>	<code>DisasterEventCard.test.tsx</code>

One rule worth restating because it's easy to drift on: files are named **domain-first, not type-first**. `disaster-event.endpoints.ts` , not `endpoints/disasterEvent.ts` sitting in a folder that already says what it is — the domain name should be readable from the

filename alone, since that's what shows up in a stack trace, an import line, and a file-search result.

17. Feature Scaffold Checklist

Every new frontend feature is built in this order. Skipping a step — especially building the page before the endpoint layer exists, or skipping the last one — is the most common way this architecture erodes.

1. Confirm the backend endpoint(s) already exist and are documented in the Backend Standards doc (section 11). Never build a frontend feature against a hoped-for API.
2. Define or confirm the domain types in `types/<domain>.types.ts`, matching the backend's `ApiResponse<T>` payload shape field-for-field.
3. Write the endpoint file (section 6).
4. Write the feature hook (section 7).
5. Build the presentational components (section 8), including data-quality handling (`<DataQualityBadge>`, "not available" fallbacks) for any field that can be OBSERVED/DERIVED/MODELLED/ESTIMATED.
6. Assemble the page (section 9) and wire its route.
7. Add Zod validation (section 11) if the feature includes a form.
8. Use the shared loading/error/empty-state components — never a bespoke one-off per feature.
9. Write Cypress E2E coverage with a fully network-stubbed spec (section 21).
10. Check every new file against the size limits (section 18).
11. Write the beginner documentation entry (section 22) — **mandatory**, mirroring the Backend Standards doc's own rule that no implementation is done until it's explained.

Worked example — the `disaster-events` feature, which is what sections 6–9 have been building throughout this doc:

```
src/types/disaster-event.types.ts
src/endpoints/disaster-event.endpoints.ts
src/hooks/useDisasterEvents.ts
src/components/disaster-events/DisasterEventCard.tsx
src/pages/disaster-events/DisasterEventsListPage.tsx
cypress/e2e/disaster-events/list.cy.ts
docs/features/disaster-events.md    ← the beginner-documentation entry,
section 22
```

That's the whole feature, in the order it gets built, with nothing skipped.

18. File Size Limits

Carried over from the source guide. A file asking to grow past its limit is a signal to split it along a real seam (extract a sub-component, split a hook's concerns, break a large endpoint file into two smaller domain files) — not a signal to raise the limit.

Layer	Limit
Page	1000 lines
Component	700 lines
Hook	150 lines
Endpoint file	100 lines
Utility	150 lines
Context	200 lines

These are self-checked during the scaffold checklist (section 17, step 10), and can be enforced mechanically later with an ESLint `max-lines` rule per directory glob if the team wants it as a CI gate rather than a discipline.

19. AI Agent Mandatory Rules

These are non-negotiable. If a request seems to require breaking one, stop and flag it rather than working around it.

1. Never call `axios / fetch / httpClient` directly from a component or page — always through a named endpoint function (section 6).
2. Never add a client-side `userId / ownerId` scoping parameter. Which data a user may see is resolved server-side, off their verified Firebase token (sections 7, 12) — this is dropped from the source guide deliberately, not an oversight.
3. Never manually read, write, or parse a Firebase ID token, and never put one in `localStorage` or a cookie. Always go through the SDK — `getIdToken()` / `getIdToken(true)` (sections 5, 15).

4. Never hardcode fake disaster events, map pins, rainfall figures, or any other data to make a screen look populated during development. An empty state (section 9) is the correct rendering of “no data yet”; a fabricated one is exactly what the project’s core data-integrity rule forbids the backend from doing — it’s forbidden here for the same reason.
5. Never fetch a full-state or full-LGA geometry payload without a bbox. Every map/geo request goes through a bbox-scoped endpoint (sections 6, 10).
6. Never render an estimated, modelled, or derived figure with the same visual weight as an observed one — use `<DataQualityBadge>` (section 8).
7. Never use `window.alert / confirm / prompt` or any native dialog — the shared `Modal / ToastContext` are the only sanctioned mechanism (section 20).
8. Never introduce a new global state store (Redux, Zustand, etc.) without a proven cross-page sharing need (section 12) — a feature hook is the default, not a store.
9. Never write a Cypress spec that touches the real API or a real database — every network call is stubbed via `cy.intercept()` (section 21).
10. Never let a file exceed its size limit (section 18) without splitting it along a real seam — never by raising the limit.
11. Never ship a feature without its beginner documentation entry (section 22).
Mandatory, no exception — mirrors the Backend Standards doc’s own rule that no implementation is finished until it’s explained.

20. No Native Browser Dialogs

Carried over from the source guide almost entirely as-is — this rule is already excellent and has nothing to do with the patient-records domain it came from.

Banned: `window.alert`, `window.confirm`, `window.prompt`, a `beforeunload` confirmation dialog, and React Router’s `unstable_usePrompt`. They block the JS event loop, can’t be styled or localized, can’t be driven or asserted on cleanly from Cypress, and are a jarring interruption on a platform meant to stay usable for an emergency responder moving fast through a form.

Replacements: `ToastContext` for a transient success/error notice (“Report submitted”, “Import failed — see errors below”); the shared `<Modal>` component for anything that needs an actual decision (confirm a delete, confirm discarding an unsaved form) — driven by ordinary component state, not a browser API.

```
function DeleteReportButton({ reportId }: { reportId: string }) {
  const [confirmOpen, setConfirmOpen] = useState(false);
  const { showToast } = useToast();

  const handleDelete = async () => {
    await deleteReport(reportId);
    setConfirmOpen(false);
    showToast({ type: "success", message: "Report deleted" });
  };

  return (
    <>
      <button onClick={() => setConfirmOpen(true)}>Delete</button>
      <Modal
        open={confirmOpen}
        title="Delete this report?"
        onCancel={() => setConfirmOpen(false)}
        onConfirm={handleDelete}
      >
        This can't be undone.
      </Modal>
    </>
  );
}
```

21. End-to-End Testing (Cypress)

Carried over from the source guide, with fixtures and examples swapped to this platform's real domain. Every spec is fully network-stubbed — a Cypress run never touches the real backend, the real database, or the real Firebase project. `data-cy` is the only sanctioned selector; never a CSS class or text match that presentation changes can silently break.

```
// cypress/e2e/disaster-events/list.cy.ts
describe("Disaster Events list", () => {
  beforeEach(() => {
    cy.login("emergency_responder"); // stubs the resolved AuthContext user,
    // never a real Firebase sign-in
    cy.interceptApi("GET", "/api/v1/disaster-events*", {
      fixture: "disaster-events/list.json",
    }).as("getDisasterEvents");
  });
});
```

```

    cy.visit("/disaster-events");
    cy.wait("@getDisasterEvents");
  });

  it("renders a card per event", () => {
    cy.dataCy("disaster-event-card").should("have.length.greaterThan", 0);
  });

  it("shows the empty state when there are no events", () => {
    cy.interceptApi("GET", "/api/v1/disaster-events*", {
      body: { success: true, data: [], message: "OK", meta: { total: 0 } },
    }).as("getEmpty");
    cy.visit("/disaster-events");
    cy.wait("@getEmpty");
    cy.contains("No disaster events recorded").should("be.visible");
  });
});

```

Custom commands live in `cypress/support/commands.ts`: `cy.login(role)` stubs the `AuthContext` 's already-resolved user for that role rather than performing a real Firebase sign-in; `cy.dataCy(selector)` is the only element-lookup helper; `cy.interceptApi(method, urlPattern, response)` wraps `cy.intercept()` with the `/api/v1` base path and the project's `ApiResponse<T>` envelope so every stub is shaped like a real response, including `meta` for paginated endpoints.

Because auth is Firebase now, not a backend session: `cy.login` must never attempt a real Firebase sign-in from a test run — that would be a live network call to a real identity provider from a CI pipeline. It stubs the resolved `AuthContext` value directly (or intercepts the `GET /api/v1/me` call section 12 depends on) so every spec runs fully offline from external services, same as every other network call.

22. Documentation & Beginner Explanations

Mandatory, mirroring the Backend Standards doc's own section 18 exactly. No frontend feature is done when the code works — it's done when a beginner on this team could read one short document and understand what was built, how it works, and where to learn the underlying concepts. The agent writes this doc as part of building the feature, not as an afterthought.

For every feature, create `docs/features/<feature-name>.md` with four parts:

1. **What was built** — a plain description of the screens/files, no jargon.
2. **How it works** — the actual technical flow, naming the specific files involved.
3. **Resources to read** — real, working links to official docs for every library or concept used, ordered the way a beginner should actually read them.
4. **Explain it like I'm new to this** — a plain-English, analogy-driven walkthrough assuming zero prior React or Firebase knowledge.

Worked example: “Signing in with Firebase”

What was built: the sign-in screen (`pages/auth/SignInPage.tsx`), `AuthContext` (section 12), a `<RequireAuth>` route wrapper, and the `useAuth()` hook every other component uses to know who's signed in.

How it works: the user types an email and password on `SignInPage.tsx`, which calls the Firebase Client SDK's `signInWithEmailAndPassword(auth, email, password)` **directly** — this goes straight to Firebase's own servers, not our backend. Firebase checks the credentials and starts a signed-in session. `AuthContext`'s `onIdTokenChanged` listener, already running app-wide (section 12), fires automatically and calls our backend's `GET /api/v1/me`, attaching the fresh Firebase ID token as a `Bearer` header. The backend verifies that token with the Firebase Admin SDK (Backend Standards doc, section 7), finds or creates the matching Postgres `users` row, and returns the resolved `role` and `scopeAdminUnitId`. `AuthContext` now holds both the Firebase identity and our app's permissions for that person, every component reading `useAuth()` re-renders as signed-in, and every future API request (`httpClient`, section 5) automatically attaches a fresh token without the sign-in screen being involved again.

Resources to read:

- Firebase Auth for Web — Get Started: <https://firebase.google.com/docs/auth/web/start>
- Firebase Auth for Web — Manage Users & ID Tokens: <https://firebase.google.com/docs/auth/web/manage-users>
- React docs — Passing Data Deeply with Context: <https://react.dev/learn/passing-data-deeply-with-context>
- Axios docs — Interceptors: <https://axios-http.com/docs/interceptors>

Explain it like I'm new to this: think of Firebase as a separate ID-checking office next door to our own building (the backend). When you sign in, you don't show your ID at our building's front desk — you walk next door first. That office checks who you are and

hands you a stamped, time-limited pass (the ID token). Every time you then walk into our building and ask for something (an API request), you show that pass at the door (the `Bearer` header). Our front desk (`authMiddleware`) checks the stamp is real and not expired. A second desk just inside (`resolveUserMiddleware`) then looks you up in our building's own visitor log — because the ID office only knows *who* you are, not *what you're allowed to do* once you're inside. In the code, `AuthContext` is the badge you're wearing that every other part of the app glances at to answer “is this person signed in, and what can they see?”

No feature is merged without this document existing for it.

23. Final Principle

Pages display. Hooks decide. Endpoints fetch. Transport handles the wire and asks Firebase for the token — it never stores one itself. Components render exactly what they're given, and never pretend a missing or estimated figure is an observed one. Maps show only what's currently in view, fetched by `bbox`, never the whole state at once. Modals and toasts replace every native dialog. Cypress proves all of it works without ever touching a real server, a real database, or a real Firebase project.

And nothing ships until there's a short, honest answer, written down, to: *what did we just build, and how would you explain it to someone new to this?*